

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



**BÁO CÁO TUẦN 2
THỰC TẬP NGÀNH KỸ THUẬT MÁY TÍNH**

**Nội Dung: XỬ LÝ DỮ LIỆU CẢM BIẾN UỖN
DẪO, THIẾT KẾ CHƯƠNG TRÌNH DỊCH CỬ
CHỈ NGÓN TAY ĐƠN GIẢN**

Giảng viên hướng dẫn: ThS. Phan Hoàng Anh
Sinh viên thực hiện: Đỗ Hoàng Gia Bảo — 23020783
Lớp: K68-E-CE1
Học phần: ELT4001

HÀ NỘI - 2026

Mục lục

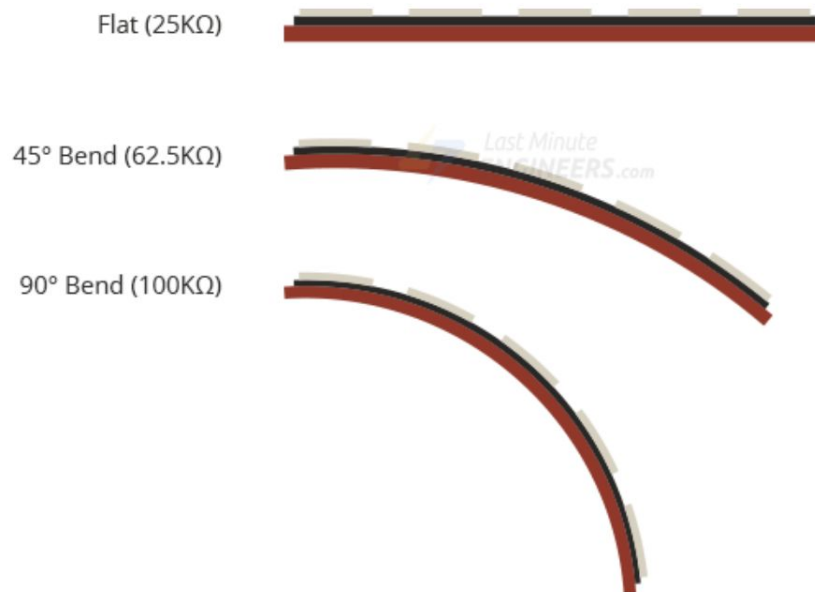
1	Đọc dữ liệu cảm biến uốn dẻo (Flex Sensor)	2
1.1	Nguyên lý hoạt động của cảm biến	2
1.2	Đọc dữ liệu cảm biến	2
1.3	Đọc dữ liệu cảm biến	3
2	Thiết kế chương trình dịch cử chỉ ngón tay	5
2.1	Thực thi chương trình	5
2.2	Kết quả mô phỏng	6

1 Đọc dữ liệu cảm biến uốn dẻo (Flex Sensor)

1.1 Nguyên lý hoạt động của cảm biến

Cảm biến uốn dẻo (Flex Sensor) là một loại cảm biến điện trở có giá trị điện trở thay đổi theo mức độ cong của cảm biến. Khi cảm biến ở trạng thái thẳng, các lớp vật liệu dẫn điện bên trong có đường dẫn dòng điện tương đối ổn định nên điện trở có giá trị nhỏ hơn. Khi cảm biến bị uốn cong, cấu trúc vật liệu dẫn điện bị kéo giãn và đường dẫn dòng điện trở nên khó khăn hơn, làm cho điện trở của cảm biến tăng lên.

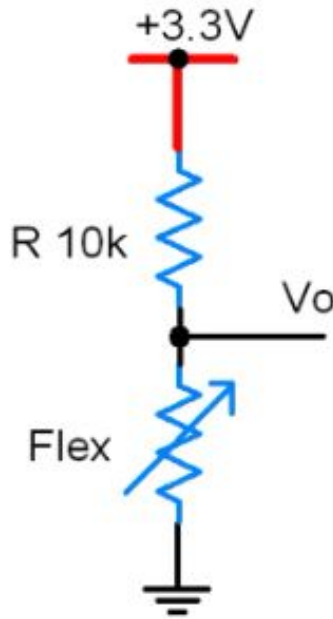
Nguyên lý hoạt động của cảm biến dựa trên hiện tượng thay đổi điện trở cơ học. Độ cong càng lớn thì sự biến dạng của lớp vật liệu dẫn điện càng mạnh, dẫn đến điện trở đo được càng cao.



Hình 1: Mối quan hệ giữa độ cong và giá trị điện trở của cảm biến uốn dẻo

1.2 Đọc dữ liệu cảm biến

Cách đơn giản nhất để đọc cảm biến uốn là kết hợp nó với một điện trở tĩnh để tạo thành mạch chia điện áp, tạo ra điện áp thay đổi có thể được đọc bởi bộ chuyển đổi tương tự sang số của vi điều khiển.



Hình 2: Sơ đồ mạch chia điện áp

Giá trị V_o được xác định bằng cách sử dụng định luật Kirchhoff's Current Law (KCL).

$$\frac{V_{CC} - V_o}{R} = \frac{V_o}{R_{Flex}}$$

$$\rightarrow V_o = \frac{V_{CC} R_{Flex}}{R + R_{Flex}}$$

Công thức mạch chia điện áp cho thấy khi cảm biến càng cong thì điện trở của nó càng tăng dẫn đến giá trị V_o càng tăng và tiệm cận đến giá trị V_{CC} .

1.3 Đọc dữ liệu cảm biến

Để vi điều khiển ESP32-WROOM đọc được dữ liệu điện thế từ mạch chia điện áp, tín hiệu V_o được đưa vào một chân ADC (Analog-to-Digital Converter) của ESP32. ADC có nhiệm vụ chuyển đổi điện áp tương tự liên tục thành một giá trị số để chương trình có thể xử lý. Với ESP32, độ phân giải ADC thường được cấu hình là 12 bit, nghĩa là giá trị đọc được nằm trong khoảng từ 0 đến 4095. Giá trị càng lớn thì điện áp đưa vào chân ADC càng cao.

Về nguyên tắc, nếu điện áp tham chiếu của ADC là V_{CC} , giá trị số đọc được từ ADC được xấp xỉ bởi công thức:

$$ADC = \frac{V_o}{V_{CC}} \times 4095$$

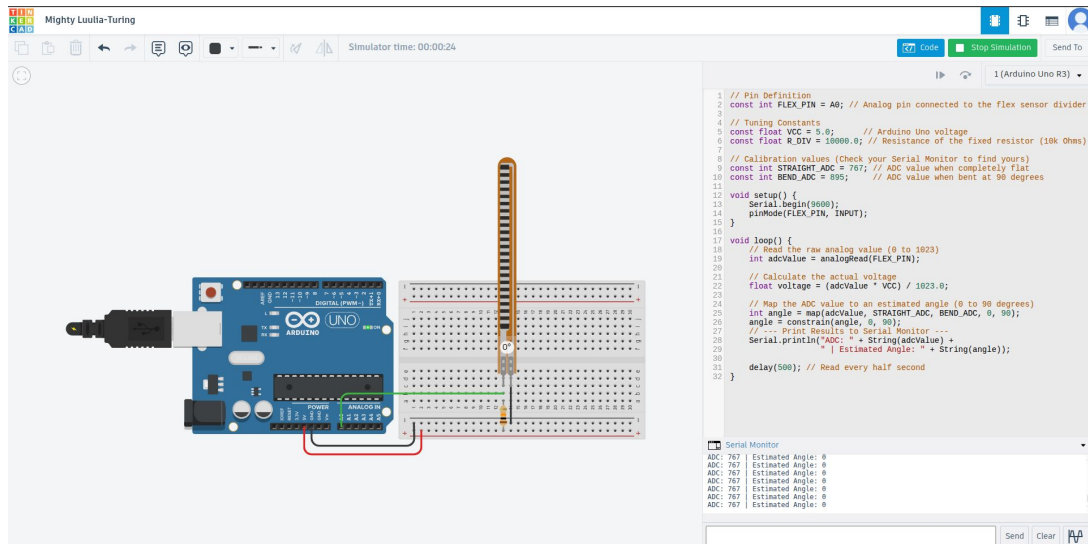
Từ đó, điện áp đầu ra của mạch chia áp có thể được tính ngược lại như sau:

$$V_o = \frac{ADC}{4095} \times V_{CC}$$

Trong thực tế, khi sử dụng ESP32-WROOM, điện áp đưa vào chân ADC không được vượt quá mức cho phép của vi điều khiển. Vì ESP32 hoạt động ở mức điện áp $3.3V$, mạch chia áp cần được thiết kế sao cho V_o nằm trong khoảng từ $0V$ đến $3.3V$. Nếu điện áp vượt quá giới hạn này, chân ADC có thể đọc sai hoặc làm hỏng vi điều khiển.

Sau khi có V_o , ta có thể đặt các ngưỡng giá trị ADC để xác định được độ cong của ngón tay. Để có thể viết chương trình dịch cử chỉ đơn giản, hiện tại phần đọc dữ liệu chỉ quan tâm một ngưỡng đặt trước. Nếu giá trị ADC vượt trên giá trị ngưỡng đó, ta suy ra ngón tay đang gấp, nếu không, ta suy ra ngón tay không gấp.

Hiện tại, cảm biến uốn dẻo vẫn chưa có để thử nghiệm với vi điều khiển ESP32 nên kết quả đang được thực hiện thông qua mô phỏng trên nền tảng [project Tinkercad](#).



Hình 3: Đọc dữ liệu cảm biến uốn dẻo trên Tinkercad

2 Thiết kế chương trình dịch cử chỉ ngón tay

2.1 Thực thi chương trình

Để đơn giản hóa bài toán dịch cử chỉ ngón tay, mỗi ngón tay được mô hình hóa chỉ với hai trạng thái: gấp và không gấp. Khi ngón tay gấp, cảm biến uốn dẻo gắn trên ngón tay đó bị cong, làm giá trị ADC đọc được tăng lên. Ngược lại, khi ngón tay không gấp, cảm biến gần như ở trạng thái thẳng nên giá trị ADC nhỏ hơn. Vì vậy, chương trình có thể sử dụng một giá trị ngưỡng cho từng ngón tay để quyết định trạng thái của ngón đó.

Gọi giá trị ADC đọc được từ cảm biến của một ngón tay là ADC_i và ngưỡng tương ứng là T_i . Trạng thái của ngón tay thứ i được xác định như sau:

$$S_i = \begin{cases} 1, & \text{nếu } ADC_i \geq T_i, \text{ ngón tay đang gấp} \\ 0, & \text{nếu } ADC_i < T_i, \text{ ngón tay không gấp} \end{cases}$$

Với cách biểu diễn này, trạng thái của mỗi ngón tay tương ứng với một bit nhị phân. Giá trị 1 biểu diễn ngón tay đang gấp, còn giá trị 0 biểu diễn ngón tay không gấp. Bàn tay có 5 ngón nên toàn bộ cử chỉ có thể được biểu diễn bằng một chuỗi 5 bit:

$$S = S_4 S_3 S_2 S_1 S_0$$

Trong đó, mỗi bit tương ứng với một ngón tay. Ví dụ, có thể quy ước S_0 là ngón cái, S_1 là ngón trỏ, S_2 là ngón giữa, S_3 là ngón áp út và S_4 là ngón út. Khi đó, một cử chỉ cụ thể của bàn tay sẽ tạo ra một mã nhị phân duy nhất. Chẳng hạn, mã 00000 có thể biểu diễn trạng thái tất cả các ngón đều không gấp, còn mã 11111 biểu diễn trạng thái tất cả các ngón đều gấp.

Do mỗi ngón tay có 2 trạng thái và có tổng cộng 5 ngón tay, số lượng tổ hợp cử chỉ khác nhau có thể biểu diễn là:

$$2^5 = 32$$

Như vậy, hệ thống có thể mã hóa tối đa 32 thông điệp khác nhau. Mỗi thông điệp được gán tương ứng với một tổ hợp trạng thái của 5 ngón tay. Ví dụ, chương trình có thể ánh xạ mã 00001 thành một thông điệp, mã 00010 thành một thông điệp khác, và tiếp tục như vậy cho đến khi xây dựng được bảng dịch cử chỉ mong muốn.

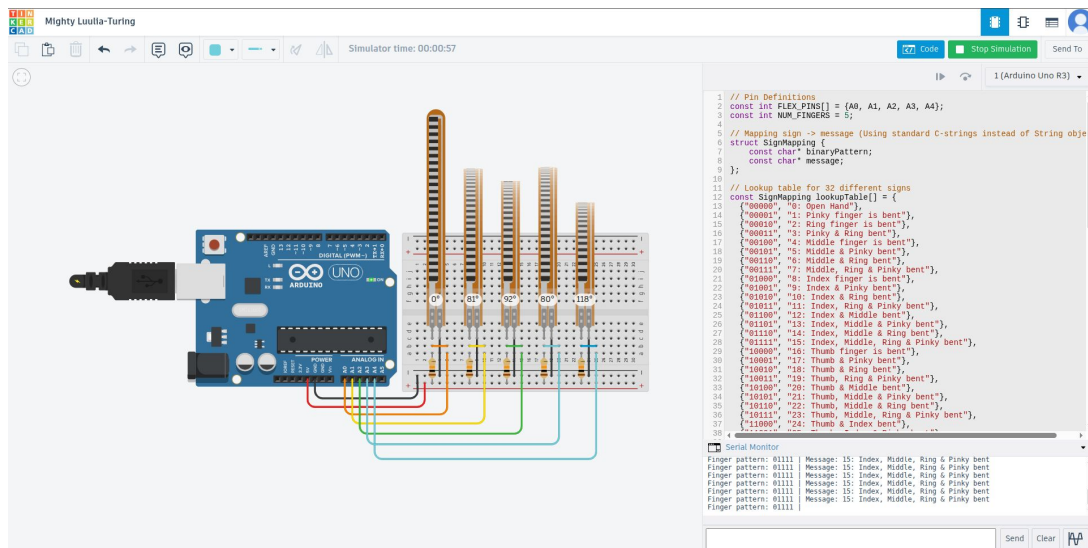
Quy trình xử lý của chương trình gồm các bước chính. Đầu tiên, ESP32 đọc giá trị ADC từ 5 cảm biến uốn dẻo tương ứng với 5 ngón tay. Tiếp theo, chương trình so sánh từng giá trị ADC với ngưỡng đã đặt trước để chuyển trạng thái của từng ngón

tay thành bit 0 hoặc 1. Sau đó, 5 bit này được ghép lại thành một mã nhị phân đại diện cho cử chỉ hiện tại. Cuối cùng, chương trình tra cứu mã nhị phân trong bảng ánh xạ để xuất ra thông điệp tương ứng.

Việc biểu diễn cử chỉ bằng mã nhị phân giúp chương trình đơn giản, dễ cài đặt và dễ mở rộng. Tuy nhiên, để hệ thống hoạt động ổn định, các giá trị ngưỡng cần được hiệu chỉnh phù hợp với từng cảm biến và từng người sử dụng, vì giá trị ADC có thể thay đổi tùy theo cách gắn cảm biến, độ cong của ngón tay và sai số của linh kiện.

2.2 Kết quả mô phỏng

Hình dưới đây thể hiện mô phỏng chương trình dịch cử chỉ ngón tay trên nền tảng Tinkercad. Do trong mô phỏng chưa sử dụng trực tiếp ESP32-WROOM, mạch được xây dựng với Arduino Uno để minh họa nguyên lý đọc tín hiệu tương tự từ các cảm biến uốn dẻo và xử lý thành thông điệp. Năm cảm biến uốn dẻo được nối với các chân analog từ A0 đến A4, tương ứng với 5 ngón tay. Mỗi cảm biến được mắc với một điện trở cố định để tạo thành mạch chia điện áp, nhờ đó vi điều khiển có thể đọc được sự thay đổi điện áp khi cảm biến bị uốn cong.



Hình 4: Mô phỏng chương trình dịch cử chỉ ngón tay trên Tinkercad

Trong mô phỏng, mỗi cảm biến được đặt ở một góc uốn khác nhau. Khi thay đổi góc uốn, giá trị điện trở của cảm biến thay đổi, kéo theo điện áp tại chân analog thay

đổi. Chương trình đọc các giá trị analog này và so sánh với ngưỡng đã đặt trước. Nếu giá trị đọc được lớn hơn ngưỡng, ngón tay tương ứng được xem là đang gập và được biểu diễn bằng bit 1. Nếu giá trị nhỏ hơn ngưỡng, ngón tay được xem là không gập và được biểu diễn bằng bit 0.

Sau khi xác định trạng thái của cả 5 ngón tay, chương trình ghép các trạng thái này thành một chuỗi nhị phân 5 bit. Chuỗi nhị phân đó được dùng làm khóa để tra cứu trong bảng ánh xạ thông điệp. Trong phần mã nguồn hiển thị ở hình, bảng ánh xạ gồm các mẫu từ 00000 đến 11111, tương ứng tối đa 32 cử chỉ khác nhau. Ví dụ, mẫu 00000 được gán với thông điệp “Open Hand”, còn các mẫu khác được gán với các tổ hợp ngón tay bị gập khác nhau.

Kết quả được hiển thị trên Serial Monitor. Ở ví dụ trong hình, Serial Monitor liên tục in ra mẫu cử chỉ đọc được là 01111 và thông điệp tương ứng là “Index, Middle, Ring & Pinky bent”. Điều này cho thấy chương trình đã đọc được trạng thái của các cảm biến, chuyển đổi thành mã nhị phân, sau đó dịch mã này thành thông điệp mô tả cử chỉ của bàn tay.

Qua mô phỏng, có thể thấy thuật toán dịch cử chỉ hoạt động theo đúng quy trình đã thiết kế: đọc dữ liệu analog từ cảm biến, chuyển thành trạng thái gập hoặc không gập, tạo mã nhị phân 5 bit và tra bảng để xuất ra thông điệp. Mô hình này tuy đơn giản nhưng phù hợp để kiểm tra nguyên lý hoạt động trước khi triển khai trên phần cứng thật với ESP32-WROOM và cảm biến uốn dẻo thực tế.